

上海交通大学试卷（A卷）

（2021 至 2022 学年 第 1 学期）

班级号_____ 学号_____ 姓名_____

课程名称_____ 操作系统_____ 成绩_____

一、内存(26')

1. 伙伴系统（buddy system）是一种高效的内存分配器算法，其本质是一种可以快速分裂和合并空闲块的分离空闲链表。
 - 1) 伙伴系统能够快速分裂和合并的重要原因可以快速定位一个块的伙伴块的位置。请简要描述根据一个块的地址和大小定位其伙伴块的方式，并计算地址为 0x20220000，大小为 32KB 的内存块的伙伴块地址。(3')
 - 2) ChCore 采用伙伴系统管理物理内存页，实现中在可用物理内存的开头划出一块连续区域存放伙伴系统的元信息（struct page 结构体数组）。假设伙伴系统管理的最小块大小为 4KB（一个页），每个物理页的元信息占 40 字节，可用物理内存从 0x100000 到 0x40000000，则实际可分配物理内存页有多少个？(3')
 - 3) 伙伴系统中最小块大小的选择，对内存管理效率有何影响？(4')
2. 虚拟内存技术的出现改变了程序员编写应用程序时的基本假设，为用户态程序提供了大量独占连续内存的假象，极大地方便了开发。
 - 1) 现代处理器支持使用多级页表配置虚拟地址到物理地址的映射，请简要描述多级页表相比单级页表带来的优势和劣势。(4')
 - 2) TLB 如何解决多级页表带来的问题？ASID 或 PCID 又解决了 TLB 的什么问题？(4')
 - 3) AArch64 架构中，通常将地址翻译过程配置为采用 4 级页表，每张页表 4KB（页表页），包含 512 个页表条目（页表项），支持的页大小为 4KB、2MB、1GB，在此配置下，请分别计算全部使用 4KB 页、全部使用 2MB 页、混合使用三种页大小这三种方式映射 0x00000000~0x66600000 地址范围所需的最少页表页数量。(4')
 - 4) AArch64 页表中有一个 nG（not Global）位，当 nG 位是 0 的时候，表示该映射所对应的 TLB 缓存项对所有 ASID 有效，反之当 nG 位是 1 的时候，表示对应的 TLB 缓存项只对当前 ASID 有效。请思考哪些情况下 nG 位应设置为 0，哪些情况下应设置为 1，两种情况下设置错了分别会有什么后果？(4')

我承诺，我将严格遵守考试纪律。

承诺人：_____

题号									
得分									
批阅人(流水阅卷教师签名处)									

二、进程和线程 (26')

1. 进程和线程是操作系统中非常重要的一组抽象。请针对以下三个场景，阐述应使用多进程或多线程，并简要分析。(3')

- 1) Web 服务器；
- 2) 文件下载；
- 3) 矩阵并行计算。

2. 多进程间的信息传递依赖于进程间通信（IPC），请列举任意两种 IPC 方式并加以对比。(4')

3. 一些轻量 Web 服务器为了减少创建进程和线程的开销，采用了纤程。为了简便，本题中纤程采用了多对一的模型，纤程的目标可以分为快速响应和后台计算两类。

- 1) 请说明操作系统使用何种技术实现打断当前进程的执行并且切换到另一个进程。(2')
- 2) 请列举使用纤程相较于使用进程的任意两个优势，对比纤程切换和进程切换的区别。（提示：从创建过程和使用场景出发）(4')
- 3) 本系统中的快速响应类纤程是否能被内核调度器感知？如果可以，请给出一种内核纤程调度的方案；如果不可以，请给出一套内核与用户态调度库的协同设计。调度目标是最小化快速响应类纤程在创建后的等待时间。(4')

4. 在 ChCore 微内核中，文件系统、磁盘驱动均被放置在用户态作为用户态服务。在宏内核设计中，文件系统和磁盘驱动处于内核态，与内核共享一个地址空间。为了简便，本题中的文件读写过程可以用“应用程序 - 文件系统 - 磁盘驱动”过程代替，假设过程中不会调用其余模块，IO 均为同步操作。

- 1) 以文件读写为例子，阐述微内核相比于宏内核的优势。（提示：可从安全性角度考虑）(3')
- 2) 根据实验，宏内核下的读写吞吐显著高于微内核下的读写吞吐。请结合 `read()` 系统调用在两种内核的调用过程分析产生这一现象的可能原因。（提示：IPC 和上下文切换）(3')
- 3) 结合你在 2) 中的分析，简要说明一种优化设计，从而可以提高微内核文件读写的吞吐。（CPU 的核心数无限多，不考虑 IO 带宽因素）(3')

三、虚拟化 (23')

1. 可虚拟化架构与不可虚拟化架构的主要区别是什么？ (3')
2. 可虚拟化架构如何处理敏感指令？不可虚拟化架构有哪些方法可以处理敏感指令（至少 2 种）？ (3')
3. 在采用设备直通的方案时，IOMMU 起到了怎样的作用？如果不用会导致什么问题？ (4')
4. 在硬件虚拟化出现之前，影子页表（Shadow Paging）被广泛用于内存虚拟化；在硬件虚拟化出现之后，则通常采用两阶段地址翻译机制。请回答以下问题：
 - 1). 第二阶段页表（Stage-2 Page Table / EPT）将哪种地址翻译为哪种地址？影子页表将哪种地址翻译为哪种地址？ (4')
 - 2). 使用影子页表时，为何要将客户操作系统页表所在内存权限设为只读？ (3')
 - 3). 使用影子页表时，如何防止客户虚拟机的应用访问客户虚拟机的内核内存？ (3')
 - 4). 假使用两阶段地址翻译时，若第一阶段与第二阶段页表均为 5 级页表，在发生 TLB 不命中时，客户虚拟地址的翻译过程总共最多需要几次访存操作？同样为 5 级页表，影子页表在 TLB 不命中的情况下，地址翻译最多需要几次访存储操作？ (3')

四、同步 (25')

MCS 锁利用了一个无锁链表将锁的竞争者插入一个锁的等待队列，锁可以顺着等待队列依次传递。锁的竞争者通过 `atomic_XCHG` 原子操作替换当前等待队尾的队尾指针 `tail` 将自己加入到等待队列中。下面展示了一个 MCS 锁的实现。

本题中可以使用的原子操作包括：

- `atomic_XCHG(addr, val)` 原子置换，返回 `addr` 中的旧值。
- `atomic_CAS(addr, compare_val, new_val)` 原子比较并替换。如果 `addr` 中的值等于 `compare_val`，则替换为 `new_val`。返回 `addr` 中的旧值。

```
struct MCS_node {
    volatile int flag;
    volatile struct MCS_node *next;
};

struct MCS_lock {
    struct MCS_node *tail;
```

```

};

void lock(struct MCS_lock *lock, struct MCS_node *me)
{
    struct MCS_node *tail = 0;
    me->next = NULL;
    me->flag = WAITING;
    tail = atomic_XCHG(&lock->tail, me);
    if (tail) {
        tail->next = me;
        while (me->flag != GRANTED)
            ;
    } else {
        me->flag = GRANTED;
    }
}

void unlock(struct MCS_lock *lock, struct MCS_node *me)
{
    if (!me->next) {
        if (atomic_CAS(&lock->tail, me, 0) == me)
            return;
        while (!me->next)
            ;
    }
    me->next->flag = GRANTED;
}

int trylock(struct MCS_lock *lock, struct MCS_node *me);
{
    ...
}

```

1. 根据以上 MCS 锁的实现，请回答以下问题：
 - 1) 当没有其他锁竞争者时（即 tail 为 NULL），me->flag = GRANTED 能否删除？为什么？ **(4')**
 - 2) 请写出 MCS 锁的 trylock 操作。如果成功上锁，则返回 0，否则返回-1。 **(3')**

2. 请思考以下问题并回答：
 - 1) 在多核场景下，为何 MCS 锁的可扩展性强于使用单一位表示上锁与否的自旋锁以及给锁竞争者编号的排号自旋锁？ **(4')**
 - 2) 自旋锁是否在某些情况下性能会优于 MCS 锁？请解释该场景。 **(4')**

3. 键值对存储引擎是将数据以键值对（Key-value）的方式存储的一个工具。其底层使用 hash 表的方式存储键值对：其会对 key 进行一次哈希（get_hash），从而找到对应的桶（bucket）中来存取对应的键值对。请回答下面问题：

```
int swap(int key1, int key2)
{
    int bucket1 = get_hash(key1); /* 获取桶编号 */
    int bucket2 = get_hash(key2);
    int ret = 0, tmp;
    /* 特殊情况处理略去，认为 bucket1 != bucket2 */
    lock(&bucket[bucket1].lock);
    lock(&bucket[bucket2].lock);
    if (bucket[bucket1].key != key1 ||
        bucket[bucket2].key != key2) {
        ret = -1; /* 没有对应的键值对 */
        goto out;
    }
    /* 交换这两个键值对的值 */
    tmp = bucket[bucket1].val;
    bucket[bucket1].val = bucket[bucket2].val;
    bucket[bucket2].val = tmp;
out:
    unlock(&bucket[bucket2].lock);
    unlock(&bucket[bucket1].lock);
    return ret;
}
```

以上代码是否会出现死锁？请解释原因，并给出一种修改的方法避免死锁。(4')

4. 非一致内存访问（NUMA）系统中跨节点的远程内存访问开销巨大。课程中介绍了一种对 NUMA 感知的互斥锁——Cohort 锁。

- 1) 请简单描述为何 Cohort 锁能够在 NUMA 系统下提供良好的性能可扩展性？与 MCS 锁相比，其有什么劣势？（可以从不同竞争程度，占用空间考虑）(4')
- 2) 如果不使用额外的每节点互斥锁，我们能否直接在给出的 MCS 锁代码中实现类似的效果（即优先传递给来自同一个 NUMA 节点的锁竞争者）？请简单描述做法。提示：可以在 MCS 节点中新加属性，如所属 NUMA 节点。(2')